

X-SAT: An Efficient Circuit-Based SAT Solver

Yuhang Qian^{1,2}, Zhihan Chen^{1,2}, Xindi Zhang^{1,2}, Shaowei Cai^{1,2,*}

¹Key Laboratory of System Software (Chinese Academy of Sciences) and State Key Laboratory of Computer Science, Institute of Software, Chinese Academy of Sciences

²School of Computer Science and Technology, University of Chinese Academy of Sciences
{qianyh, chenzh, zhangxd, caisw}@ios.ac.cn

Abstract—In modern digital circuit design, verifying the equivalence of arithmetic circuits is a significant and challenging task. This paper introduces a new circuit solver based on the Conflict-Driven Clause Learning (CDCL) algorithm, which integrates structural elimination techniques to reduce the number of variables and clauses while maintaining the circuit structure. Additionally, branching heuristics have been enhanced specifically for the structure of arithmetic circuits. Experimental results demonstrate that X-SAT significantly outperforms best previous circuit solver could be found on all benchmarks. Further, X-SAT performs better than the state-of-the-art CNF-based SAT solvers on complex arithmetic circuits, underscoring its significant potential in the field of circuit design verification.

Index Terms—Boolean satisfiability, Equivalence checking, Circuit-based SAT, EDA.

I. INTRODUCTION

Boolean Satisfiability (SAT) is a fundamental problem in computer science with wide-ranging applications, especially in electronic design automation (EDA). Logic equivalence checking (LEC) [1], logic synthesis [2], bounded model checking, and automatic test pattern generation (ATPG) [3] commonly rely on SAT solvers. Typically, circuits are encoded into conjunctive normal form (CNF) for SAT solving.

Modern CNF solvers [4]–[6] are based on the Conflict Driven Clause Learning (CDCL) method, and equipped with numerous powerful and efficient techniques, including the two-literal watching scheme [7] for efficient Boolean Constraint Propagation (BCP), Bounded Variable Elimination (BVE) [8], variable selection heuristics such as VSIDS [7], learnt clause reduction, among others. However, these strategies and optimizations are designed based on the CNF data structure and do not take into account any information from the circuit.

When solving problems arising from EDA tasks, the transformation to CNF discards the structural information of the original circuit, limiting optimization potential. To address this issue, QuteSAT [9] improves the BCP process, enabling propagation directly on circuits and supporting both simple and complex gates. Additionally, the solver supports a circuit-structure-based branching strategy called **J-frontier**. Zhang et al. [2] introduced a SAT solver tailored for logic synthesis, which was built upon the CNF-based solver Glucose [5]. Besides performing BCP in circuit format, it also employs improved data structures to maintain the J-frontier branching heuristic. Hu et al. [10] used logic synthesis techniques for circuit rewriting before SAT solving.

This paper presents a novel circuit SAT solver that is lightweight and efficient, with the focus on arithmetic circuits. Our contributions are summarized as follows:

- We propose a structural elimination technique to simplify the circuits before SAT solving. Specifically, we reduce variables by identifying XOR gates and synthesizing non-XOR gates into multi-input gates. This technique is effective for both arithmetic and non-arithmetic circuits. It also improves the search efficiency of the CDCL framework through a more compact circuit representation and distinguishes our method from traditional solvers that operate directly on the original circuit structure.
- We design a new branching heuristic method based on VSIDS [7] called **XVSIDS**. During conflict analysis, we increase the branching weights of neighboring XOR gates for the analyzed variables, enhancing the heuristic's locality. It has achieved good performance on arithmetic circuits, more robust than the J-Frontier heuristic.
- We develop a circuit SAT solver based on the CDCL framework, which integrates these two techniques. Our circuit SAT solver has significantly outperformed the best previous circuit solver we could find on both arithmetic and non-arithmetic circuits. Moreover, our solver surpasses the state-of-the-art CNF SAT solver Kissat [4] on arithmetic circuits, noting that Kissat and its variants dominates the main tracks of recent SAT competitions from 2020. This is the first time that a circuit SAT solver is compared to state-of-the-art CNF SAT solver (previous works usually compared to MiniSAT [6] or Glucose [5]) and achieves better results.

The rest of the paper is organized as follows: Section II introduces the background on SAT solving, AIG graph, circuit-based SAT solving and CEC problem. Section III presents the main methods used in the design of our solver, including the motivating example, solving framework and the implementation of structural elimination and the XVIDS branching strategy. Section IV presents experimental results while Section V concludes the paper.

II. BACKGROUND

A. Boolean Satisfiability

The Boolean Satisfiability (SAT) asks whether there is an assignment to the variables of a Boolean formula in Conjunctive Normal Form (CNF) that makes it true. A literal is either

*Corresponding author

Algorithm 1: Basic framework of CDCL Algorithm

```
Input: CNF formula  $F$ 
Output: SAT or UNSAT
 $\alpha = \{\}$ ;
 $dl \leftarrow 0$ ;
if  $\text{BooleanUnitPropagation}(F, \alpha) == \text{CONFLICT}$ 
  then
    return UNSAT ;
end
while  $\exists$  unassigned variables do
   $(x, v) \leftarrow \text{SelectBranchVar}(F, \alpha)$  ;
   $dl \leftarrow dl + 1$  ;
   $\alpha \leftarrow \alpha \cup \{(x, v)\}$  ;
  if  $\text{UnitPropagation}(F, \alpha) == \text{CONFLICT}$  then
     $bl \leftarrow \text{ConflictAnalysis}(F, \alpha)$  ;
    if  $bl < 0$  then
      return UNSAT ;
    end
    else
       $\text{Backtrack}(F, \alpha, bl)$  ;
       $dl \leftarrow bl$  ;
    end
  end
end
return SAT ;
```

a Boolean variable or its negation. A formula is represented as a conjunction of clauses, where each clause is a disjunction of literals:

$$\phi = (C_1 \wedge C_2 \wedge \dots \wedge C_m), \quad C_i = (l_{i1} \vee l_{i2} \vee \dots)$$

The goal is to find an assignment $\mathbf{x}$ such that $\phi(\mathbf{x}) = 1$ or prove none exists. SAT is NP-complete, making it computationally challenging for large instances, while it remains a fundamental problem in hardware verification.

B. SAT Solving Framework

Modern SAT solvers, such as MiniSAT, Glucose, and Kissat are based on the Conflict-Driven Clause Learning (CDCL) framework [11]. Key techniques in CDCL-based SAT solvers, including efficient Boolean Constraint Propagation (BCP), conflict analysis, clause learning, rephasing, restarts, learnt clause reduction, and Bounded Variable Elimination (BVE), have significantly enhanced their performance in practical applications.

The pseudo-code in Algorithm 1 represents the basic framework of the Conflict-Driven Clause Learning (CDCL) algorithm. The key functions include **BooleanUnitPropagation**(F, α), which propagates unit clauses in the formula F based on the partial assignment α and returns ‘CONFLICT’ if a contradiction occurs; **SelectBranchVar**(F, α), which selects an unassigned variable x and assigns it a value v ;

ConflictAnalysis(F, α), which analyzes conflicts and determines the backtrack level bl ; and **Backtrack**(F, α, bl), which reverts the assignment to the backtrack level bl . The algorithm starts with an empty partial assignment and decision level, iteratively selects variables, propagates unit clauses, and uses conflict analysis and backtracking to resolve conflicts. It terminates when a satisfiable solution is found (SAT) or the formula is unsatisfiable (UNSAT).

C. And-Inverter Graph

And-Inverter Graph (AIG) is a representation of Boolean logic circuits that uses AND gates and inverters (NOT gates) as the basic components. AIGs are a popular choice for circuit representations because they provide an efficient way to model and manipulate Boolean functions. AIGs are particularly useful for circuit verification, as they allow for efficient manipulation of logic functions and can be easily transformed into Conjunctive Normal Form (CNF) for SAT solvers.

D. Circuit-based SAT Solving

Circuit-based SAT solving shares the same CDCL framework as SAT solvers for CNF formulas, with most techniques being interchangeable. The primary differences lie in how decisions and propagations are performed. In circuit solvers, decisions are made directly on wires in the circuit structure, and assignments are propagated through the circuit, rather than through the CNF representation as in conventional SAT solvers. This approach eliminates the overhead of converting circuits into CNF, making circuit solvers more efficient for scenarios requiring repeated invocations. Additionally, circuit solvers can employ structure-aware branching heuristics, such as the J-frontier heuristic [9], to improve decision-making and enhance solving efficiency.

E. CEC Problem

Combinational Equivalence Checking (CEC) is the problem of determining whether two Boolean circuits or logic functions are functionally equivalent. It is a fundamental task in EDA with applications in formal verification, circuit optimization, and hardware testing. To perform CEC, the two circuits are first combined into a *miter circuit*, where the inputs of both circuits are shared, and their outputs are XORed. The output of the miter circuit is used to detect discrepancies: if it is always 0 for all input combinations, the circuits are equivalent; otherwise, they are not.

Once the miter circuit is constructed, the problem is encoded as a SAT instance and solved using a SAT solver. A key optimization is *SAT-Sweeping* [12], [13], which iteratively calls a SAT solver to verify the equivalence of potentially equivalent internal nodes, and merges equivalent nodes to reduce complexity.

III. PROPOSED METHOD

A. Motivating Example

Although AIG provides a convenient expression for circuits, it is usually too basic for complex circuits such as arithmetic

circuits. By identifying the structure and re-write the circuits using more compact gates, we can significantly reduce the number of variables in the circuits, making the circuit SAT solving faster.

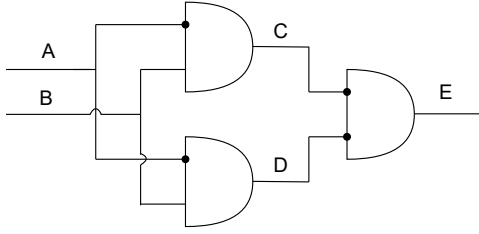


Fig. 1. An example of a small subcircuit in AIG format

As illustrated in Figure 1, the given circuit demonstrates a scenario where the value of E is determined by inputs A and B . In an And-Inverter Graph (AIG), due to its limited expressive power, additional variables C and D must be introduced to construct the complete circuit. However, an alternative approach is to directly construct a 2-input LUT gate with A and B as inputs and E as the output. More generally, by merging the circuit into multi-input LUT gates, it is possible to significantly reduce the number of variables while maintaining the overall structure of the circuit. This observation inspired the development of the **Structural Elimination** method, which will be discussed in detail in the following sections.

Furthermore, XOR gates exhibit unique properties that can be leveraged in circuit analysis. Building upon the VSIDS heuristic, we introduce specific enhancements tailored for XOR gates, leading to a new heuristic named **XVSIDS**.

B. Overview of Our Circuit-based SAT Method

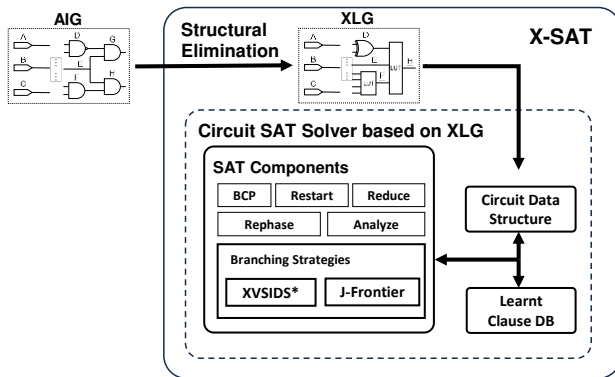


Fig. 2. Main framework of X-SAT

We develop an efficient circuit-based SAT solver. The solver is composed of two parts, a rewriting procedure and a CDCL-based solver.

In contrast to other circuit solvers that operate directly on AIG circuits, our method transforms AIG circuits into XLG

circuits, preserving the circuit structure while reducing variables. This lightweight approach is particularly advantageous for iterative and frequent invocations, making our solver well-suited for SAT-sweep processes and model checking. It is particularly effective for arithmetic circuits.

For SAT solving, we introduce the XVIDS branching strategy, adapted from VSIDS and optimized for the compact XLG circuit representation. For comprehensive benchmarking, we also incorporate a j-frontier branching heuristic, similar to those utilized in other circuit solvers, to enable direct and meaningful comparisons.

C. Circuit Structural Elimination

Our method first identifies the binary XOR relationships in the AIG and then rewrites the original AIG circuit by incorporating XOR gates to express the circuit. This type of circuit is referred to as Xor-And-Inverter Graph (XAG).

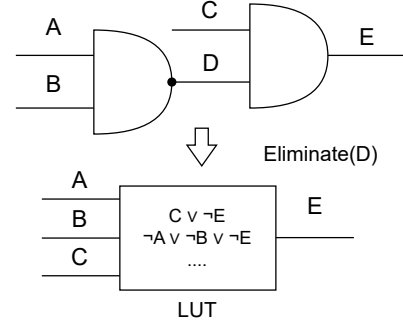


Fig. 3. An example of the elimination process

In addition to XOR gates, the non-XOR parts of the circuit could be further simplified. We merge binary gates into multi-input gates to simplify the circuit. The logical relationships between the inputs and outputs of multi-input gates can be expressed using CNF formulas, which have greater expressive power and are better suited for the CDCL solving framework. As shown in Figure 3, we eliminate the variable D and express the relationship between the inputs and the output of the introduced three-input gate using a CNF formula. Here we borrow a concept from FPGA and refer to the merged multi-input gates as LUTs. Such multi-input gates, represented in CNF, can express any relationship between inputs and outputs, rather than being restricted to well-known logical circuit gates such as multi-input AND, multi-input XOR, or MUX. Ultimately, we obtain a graph that includes both LUTs and XOR gates, which we refer to as the the Xor-Lut-Inverter Graph (XLG).

The selection of variables for elimination and the order in which they are eliminated have significant impact to the algorithm's performance. Our algorithm relies on a concept called *elimination score*, which measures the change on the number of literals, noting that some literals would be deleted while new literals are introduced when a variable is eliminated.

Algorithm 2: Circuit Structural Elimination

Input: AIG format circuit A
Output: XLG format circuit L

```
1  $xag \leftarrow$  identify xor gates and rewrite  $A$ ;  
2  $heap \leftarrow$  a max heap sorted by the elimination score  
   associated with each variable;  
3  $L \leftarrow xag$ ;  
4 while  $heap$  is not empty do  
5    $gate \leftarrow$  gate with max elimination score in  $heap$   
6   if  $gate$  is not a single fan-out gate then  
7     continue;  
8   end  
9    $fanout\_gate \leftarrow$  single fanout gate of  $g$ ;  
10  if  $gate$  or  $fanout\_gate$  is XOR gate then  
11    continue;  
12  end  
13   $new\_input\_num \leftarrow$  fan-in numbers of  $gate$  +  
   fan-in numbers of  $fanout\_gate$  - 1  
14  if  $new\_input\_num > \theta$  then  
15    continue;  
16  end  
17   $L \leftarrow$  perform elimination for  $gate$  in  $L$   
18  update new elimination score for fan-in and fan-out  
   gates of  $gate$   
19 end
```

This transformation method is described in Algorithm 1. Initially, we identify the XOR gates and construct an XAG (XOR-AIG) graph (line 1). Next, we compute the elimination scores for all gate variables and maintain a max heap for efficient access (line 2). During each iteration, we select the gate variable with the largest score from the heap for elimination, but those gates with more than one fanout or the inputs of the newly merged gate exceeds a predefined limit (denoted as θ) are not eligible. Based on our observations, setting θ to 7 proves to be effective. When a gate variable is eliminated, the elimination scores of adjacent gate variables are affected and will be recalculated (lines 4-19). At this stage, the XAG is successfully simplified into an XLG graph composed of LUT and XOR gates.

D. XVSIDS: A Branching Heuristic Based on Circuit Structure

SAT solvers usually do not perform well on instances with a large amount of XORs, because the CDCL framework is not well-suited for handling XOR information effectively.

The traditional VSIDS algorithm updates the variable scores during conflict analysis by incrementing the scores of all the variables involved in the conflict [14]. To optimize this process for XOR gates in circuits, we propose a novel branching heuristic called **XVSIDS**.

After converting the circuit into its XOR Logic Graph (XLG) representation, we define “XOR edge” variables as those corresponding to XOR gates or directly influenced by XOR gates in their fan-in or fan-out. Since there are many

“XOR edge” variables in arithmetic circuits, most variables have XOR neighboring variables.

To leverage this structure, our XVSIDS strategy modifies the variable score bumping mechanism: when a variable’s score is increased, its adjacent XOR gates are also increased with a increment of half value of the primary variable’s increment. This adjustment aims to enhance the locality of the CDCL search by prioritizing variables that are structurally significant in arithmetic circuits. Experimental results demonstrate that this approach yields substantial performance improvements.

IV. EXPERIMENTAL RESULTS

In this section, we present the experimental setup, dataset sources and solvers in our experiments. Then, we report the evaluation results and perform experimental analysis of the proposed ideas.

A. Experimental Settings

We implement our X-SAT solver in C++ language and compile it with GNU g++ (version 10.5.0). It can be accessed from our GitHub repository ¹. All tools and competitors are written in C/C++ and compiled in the same environment as X-SAT.

All experiments were conducted on a high-performance computing cluster equipped with two AMD EPYC 7763 CPUs @ 2.45 GHz, totaling 128 physical cores, and 1TB RAM. The system runs Ubuntu 20.04 LTS (64-bit). A single core and less than 10GB was used for any test case considered in this section.

To validate and compare the performance of the solvers, we use Combinational Equivalence Checking (CEC) problems for testing. The dataset for testing is categorized into two groups: arithmetic circuits and non-arithmetic circuits, all in AIG format. The dataset can also be accessed through GitHub ². To enable the CNF solver to solve the AIG, we used the AIGER tool [15] to convert them into CNF format.

- **Arithmetic Circuits:** We use 22 industrial instances with the application of long bit-wise optimization, which are datapath circuits with adders and multipliers. Both SAT and UNSAT circuits are included. The vast majority are UNSAT instances, with b27 being the only SAT instance.
- **Non-Arithmetic Circuits:** We select 13 academic miter circuits which are generated from the ISCAS85, ISCAS89, and ITC99 benchmark suites, emphasizing circuits of moderate complexity for demonstration. The method for generating miter circuits involves first converting the circuit into a combinational circuit using scan chains. Then, logic synthesis is performed using the ABC tool [16] before creating the miter circuit. The logic synthesis commands [17], `&st;` `&dch;` `&if -K 6;` `&mfs;`, are executed three times iteratively. This sequence is executed three times iteratively. Specifically, the selection criterion is based on the solving time of the

¹<https://github.com/shaowei-cai-group/X-SAT>

²<https://github.com/shaowei-cai-group/X-SAT/tree/main/data>

moderately capable Glucose solver. Instances are selected if they can be solved within the time limit (2000 seconds) and have a solving time greater than 1 second.

B. Evaluation Metrics and Solvers

We evaluated our proposed solver, X-SAT, against three state-of-the-art CNF solvers and one advanced circuit solver.

- **abc-cirsat** [2]: The latest advanced circuit SAT solver, integrated into ABC. The official name of the solver could not be found, so it is referred to as abc-cirsat in this paper.
- **kissat** [4]: A state-of-the-art CNF SAT solver that dominated the main track of the 2020, 2021, 2022 and 2024 SAT competition. The latest version kissat-sc2024 is selected.
- **glucose** [5]: A widely-used SAT solver that improves upon several techniques introduced by MiniSat.
- **minisat** [6]: A SAT solver widely-used in industries, frequently used as a benchmark for performance evaluation.

Our X-SAT solver and abc-cirsat belong to circuit based SAT solver, while kissat, glucose and minisat belong to CNF based SAT solvers.

The performance metrics are summarized using PAR2 (penalized average runtime). If a solver fails to solve a case within the 2000-second time limit, its runtime is recorded as 4000 seconds; otherwise, the actual runtime is used.

C. Results and Analysis

Table I summarizes the performance results for arithmetic and non-arithmetic circuits. It provides the following details for each instance: instance name, gate count, and execution times for each solver: kissat, glucose, minisat, abc-cirsat and x-sat.

Arithmetic Circuits: The results show that our proposed X-SAT solver, utilizing the structural elimination technique with the XVSIDS branching strategy, achieves an **38.26x** speedup compared to the other circuit SAT solver abc-cirsat. Furthermore, it delivers a **1.36x** improvement over the state-of-the-art CNF solver kissat.

Non-Arithmetic Circuits: X-SAT achieves a 7.08x speedup over abc-cirsat, while 2.01x slower than the best CNF based SAT solver kissat on non-arithmetic circuits. However, X-SAT still outperforms glucose by 3.19x and minisat by 17.45x. For circuit-based SAT solvers, we have significantly reduced the performance gap with kissat for the first time.

D. Validation of Effectiveness

As shown in Table II, we compared two different solver configurations: one with structural elimination disabled (No-elim) and one with structural elimination enabled (Elim). For each configuration, we tested three different branching heuristics (J-Frontier, VSIDS, XVSIDS) and recorded the solving time for each instance.

Structural Elimination: Compared to circuits in AIG format, the XLG format reduces the number of variables by an average of 61.39% for arithmetic circuits and 61.90% for non-arithmetic circuits. For arithmetic circuits, enabling structural

TABLE I
COMPARING WITH OTHER SOLVERS (CUTOFF=2000s)

Instance	#Gates	CNF SAT			Circuit SAT	
		kissat	glucose	minisat	abc-cirsat	x-sat
ARITHMETIC CIRCUIT						
aa1	17940	103.41	—	—	—	123.04
aa30	17948	63.80	1678.14	—	—	36.72
aa38	17998	75.43	1759.38	—	—	74.57
ab18	19355	105.75	—	—	—	116.00
ab1	19354	75.95	958.29	—	—	45.39
ab38	19359	91.50	1451.84	—	—	76.94
ab40	19255	99.29	801.23	—	—	34.42
ac1	21792	187.40	—	—	—	181.62
ac20	21797	100.50	1978.48	—	—	68.17
ac2	21511	69.66	839.78	—	696.84	44.44
ac33	21024	69.34	331.14	—	399.13	30.28
ac3	21812	130.44	1798.88	—	—	140.52
ac40	21668	110.81	936.67	—	—	52.55
ac42	21286	68.99	463.62	—	408.42	34.75
ad14	23307	168.03	—	—	—	239.45
ad1	23301	103.22	1190.55	—	—	62.21
ad21	23035	90.71	466.48	—	826.44	39.01
ad2	22527	80.46	384.01	—	474.50	36.58
ad3	23214	115.58	650.99	—	—	51.85
ad41	22795	90.98	438.55	—	413.26	38.12
ad44	23319	172.05	1591.62	—	—	120.26
b27	12372	75.32	31.18	345.88	63.26	7.20
PAR2		102.21	1534.13	3833.90	2876.45	75.19
NON-ARITHMETIC CIRCUIT						
c3540	2113	0.86	4.43	23.77	1.08	0.31
c5315	3636	0.34	1.37	8.76	0.31	0.26
c7552	3803	0.33	1.16	2.50	21.09	0.32
b14	12033	3.83	39.21	291.02	170.16	8.33
b15	18212	3.81	13.35	20.62	24.35	4.22
b17	57076	43.11	145.42	258.24	203.98	74.47
b20	25255	7.88	103.22	1003.84	341.60	21.60
b21	25556	8.73	112.77	907.46	101.66	22.32
b22	37903	20.68	161.86	728.57	446.56	39.89
s15850	7978	0.51	1.24	2.48	1.22	0.63
s35932	19332	0.57	1.25	1.56	1.77	2.55
s38417	18703	0.94	4.44	7.63	4.65	3.39
s38584	26603	1.82	8.54	11.94	8.27	9.03
PAR2		7.18	46.02	251.41	102.05	14.41

TABLE II
DIFFERENT VERSIONS OF X-SAT (CUTOFF=2000s)

Instance	No-elim			Elim		
	J-Frontier	VSIDS	XVSIDS	J-Frontier	VSIDS	XVSIDS
ARITHMETIC CIRCUIT						
aa1	–	442.25	219.69	–	328.81	123.04
aa30	–	190.38	70.47	522.18	133.16	36.72
aa38	–	333.38	118.06	1634.51	257.32	74.57
ab18	–	549.97	230.01	–	371.90	116.00
ab1	–	140.14	80.67	632.14	98.17	45.39
ab38	–	266.47	116.98	1849.23	260.87	76.94
ab40	818.54	127.96	62.84	149.68	84.35	34.42
ac1	–	815.14	364.85	–	664.78	181.62
ac20	–	265.03	113.90	598.81	190.03	68.17
ac2	532.56	122.45	69.65	219.89	104.68	44.44
ac33	611.07	119.07	55.16	114.22	91.93	30.28
ac3	–	576.35	198.07	–	444.73	140.52
ac40	1112.11	156.90	97.32	275.65	146.45	52.55
ac42	530.51	147.21	56.13	250.23	81.31	34.75
ad14	–	730.88	347.58	–	619.14	239.45
ad1	–	182.38	144.67	690.73	130.09	62.21
ad21	799.64	126.95	82.50	307.20	88.92	39.01
ad2	580.60	78.72	58.54	180.11	65.86	36.58
ad3	1097.23	129.79	112.38	291.07	100.50	51.85
ad41	530.77	103.88	64.91	299.33	84.31	38.12
ad44	–	439.39	194.15	–	313.12	120.26
b27	51.80	21.09	61.86	55.59	32.30	7.20
PAR2	2484.77	275.72	132.75	1457.75	213.31	75.19
NON-ARITHMETIC CIRCUIT						
c3540	0.92	0.50	0.65	0.59	0.38	0.31
c5315	0.38	0.31	0.32	0.29	0.26	0.26
c7552	0.56	0.41	0.43	0.31	0.33	0.32
b14	27.72	10.73	16.72	30.18	7.61	8.33
b15	17.21	4.47	11.62	10.95	3.45	4.22
b17	370.28	125.66	111.01	182.40	88.22	74.47
b20	90.48	24.37	24.22	73.84	13.11	21.60
b21	134.85	24.45	30.28	46.19	14.41	22.32
b22	283.39	53.48	59.21	93.35	31.30	39.89
s15850	2.37	1.08	0.79	1.11	0.67	0.63
s35932	7.88	7.30	5.72	3.93	3.11	2.55
s38417	20.52	6.03	5.35	8.32	3.64	3.39
s38584	38.02	17.32	11.03	20.36	8.57	9.03
PAR2	76.51	21.24	21.33	36.29	13.47	14.41

elimination resulted in improvements of 1.70x, 1.29x, and 1.77x for the three heuristics(J-Frontier, VSIDS, XVSIDS), respectively. For non-arithmetic circuits, the improvements were 2.11x, 1.58x, and 1.48x. This demonstrates that structural elimination is a fundamental optimization, providing a positive impact across all circuits and heuristics.

XVSIDS: With structural elimination enabled, XVSIDS outperforms the traditional J-Frontier branching method, achieving a 19.39x improvement on arithmetic circuits and a 2.52x improvement on non-arithmetic circuits. This demonstrates that XVSIDS has a clear advantage over the traditional method. Similarly, with structural elimination enabled, XVSIDS achieves a 2.84x improvement over VSIDS on arithmetic circuits. On non-arithmetic circuits, the two methods show little difference, with PAR2 values of 13.47(VSIDS) and 14.41(XVSIDS), respectively.

V. CONCLUSION

In this paper, we proposed an efficient circuit SAT solver, named X-SAT. We introduced elimination strategies to rewrite the AIGs into a new format, XLG, which reduces variables. Additionally, we improved the traditional VSIDS strategy by considering XOR gates, leading to the XVSIDS heuristic, which demonstrated substantial performance gains for both arithmetic and non-arithmetic circuits.

X-SAT marks a significant milestone in circuit SAT solving by being the first solver to outperform the state-of-the-art CNF solver, Kissat, on arithmetic circuits. Moreover, its performance on non-arithmetic circuits remains competitive with the most advanced CNF SAT solvers, demonstrating its versatility and robustness across diverse circuit types.

Future work includes extending X-SAT's application to EDA tasks and designing a parallel circuit SAT solver to address even more complex and computationally demanding problems.

ACKNOWLEDGMENT

This work was supported by the Strategic Priority Research Program of the Chinese Academy of Sciences, Grant No. XDA0320000 and XDA0320300.

REFERENCES

- [1] E. I. Goldberg, M. R. Prasad, and R. K. Brayton, "Using sat for combinational equivalence checking," in *Proceedings Design, Automation and Test in Europe. Conference and Exhibition 2001*. IEEE, 2001, pp. 114–121.
- [2] H.-T. Zhang, J.-H. R. Jiang, and A. Mishchenko, "A circuit-based sat solver for logic synthesis," in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2021, pp. 1–6.
- [3] P. Stephan, R. K. Brayton, and A. L. Sangiovanni-Vincentelli, "Combinational test generation using satisfiability," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 15, no. 9, pp. 1167–1176, 1996.
- [4] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, and F. Pollitt, "Cadical, gimsatul, isasat and kissat entering the sat competition 2024," *SAT COMPETITION 2024*, p. 8, 2024.
- [5] G. Audemard and L. Simon, "On the glucose sat solver," *International Journal on Artificial Intelligence Tools*, vol. 27, no. 01, p. 1840001, 2018.
- [6] N. Een, "Minisat-a sat solver with conflict-clause minimization," *Proc. Theory and Applications of Satisfiability Testing (SAT 05)*, 2005.
- [7] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, and S. Malik, "Chaff: Engineering an efficient sat solver," in *Proceedings of the 38th annual Design Automation Conference*, 2001, pp. 530–535.
- [8] N. Manthey, "Coprocesor 2.0—a flexible cnf simplifier: (tool presentation)," in *Theory and Applications of Satisfiability Testing—SAT 2012: 15th International Conference, Trento, Italy, June 17–20, 2012. Proceedings 15*. Springer, 2012, pp. 436–441.
- [9] C.-A. Wu, T.-H. Lin, C.-C. Lee, and C.-Y. Huang, "Qutesat: a robust circuit-based sat solver for complex circuit structure," in *2007 Design, Automation & Test in Europe Conference & Exhibition*. IEEE, 2007, pp. 1–6.
- [10] K. Hu and Z. Chu, "An efficient circuit-based sat solver and its application in logic equivalence checking," *Microelectronics Journal*, vol. 142, p. 106005, 2023.
- [11] J. M. Silva and K. A. Sakallah, "Grasp-a new search algorithm for satisfiability," in *Proceedings of International Conference on Computer Aided Design*. IEEE, 1996, pp. 220–227.
- [12] Q. Zhu, N. Kitchen, A. Kuehlmann, and A. Sangiovanni-Vincentelli, "Sat sweeping with local observability don't-cares," in *Proceedings of the 43rd Annual Design Automation Conference*, 2006, pp. 229–234.
- [13] A. Mishchenko, S. Chatterjee, R. Brayton, and N. Een, "Improvements to combinational equivalence checking," in *Proceedings of the 2006 IEEE/ACM international conference on Computer-aided design*, 2006, pp. 836–843.
- [14] A. Biere and A. Fröhlich, "Evaluating cdcl variable scoring schemes," in *Theory and Applications of Satisfiability Testing—SAT 2015: 18th International Conference, Austin, TX, USA, September 24–27, 2015, Proceedings 18*. Springer, 2015, pp. 405–422.
- [15] A. Biere, K. Heljanko, and S. Wieringa, "AIGER 1.9 And Beyond," Institute for Formal Models and Verification, Johannes Kepler University, Altenbergerstr. 69, 4040 Linz, Austria, Tech. Rep. 11/2, Jul. 2011.
- [16] "ABC," <https://github.com/berkeley-abc/abc>.
- [17] H.-T. Zhang, J.-H. R. Jiang, L. Amarú, A. Mishchenko, and R. Brayton, "Deep integration of circuit simulator and sat solver," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 877–882.